Cambridge Style Pseudocode: AJ_24038155 Dialect v1.0

1. Language Overview

The AJ_24038155 Dialect is a structured pseudocode language intended for:

- representing algorithms generated from natural-language tasks;
- deterministic parsing into an Abstract Syntax Tree;
- conversion into a Control Flow Graph;
- visual generation of flow diagrams;
- deterministic translation into Python;
- safe behavioural evaluation.

The language uses explicit block terminators and does not rely on indentation for program structure.

Keywords are case-insensitive during parsing but are presented in uppercase as the canonical format.

Identifiers remain case-sensitive.

---

2. Lexical Rules

2.1 Identifiers

An identifier must begin with a letter or underscore.

```
identifier ::= letter , { letter | digit | "_" } ;
```

Examples:

```
total
item_count
_grid
value2
```

Invalid examples:

```
2value
item-count
total value
```

2.2 Integer Literals

```
integer_literal ::= [ "-" ] , digit , { digit } ;
```

Examples:

```
0
12
-5
```

1000

2.3 Float Literals

float_literal ::= ["-"] , digit , { digit } , "." , digit , { digit } ;

Examples:

0.5
3.14
-10.25

2.4 String Literals

Strings use the STRING constructor with quoted contents.

string_literal ::= "STRING" , "(" , quoted_string , ")" ;

Examples:

STRING("Hello")
STRING("Enter a number")
STRING("The result is {result}")

Escape sequences may be used:

\"
\\
\n
\t

Example:

STRING("He said \"Hello\"")

2.5 Character Literals

Characters use the CHAR constructor and must contain exactly one character or one valid escape sequence.

char_literal ::= "CHAR" , "(" , quoted_character , ")" ;

Examples:

CHAR("a")
CHAR("7")
CHAR("\n")

2.6 Boolean Literals

boolean_literal ::= "BOOLEAN" , "(" , ("TRUE" | "FALSE") , ")" ;

Canonical examples:

BOOLEAN(TRUE)
BOOLEAN(FALSE)

2.7 Comments

Single-line comments begin with COMMENT.

comment ::= "COMMENT" , comment_text , newline ;

Example:

COMMENT Calculate the running total

Comments are ignored during AST construction but may be stored as metadata.

2.8 Whitespace and Newlines

Spaces and indentation are not semantically significant except inside string literals.

Each statement must end with a newline.

Block structure is determined by explicit keywords such as:

ENDIF
ENDWHILE
ENDFOR
ENDFUNCTION

---

3. Operator Rules

3.1 Assignment

Assignment uses <-.

variable <- value

Example:

total <- 0
name <- STRING("Alan")

3.2 Arithmetic Operators

+
-

*
/
INTEGER DIVISION
MODULUS
EXP

Examples:

```
sum <- a + b
difference <- a - b
product <- a * b
quotient <- a / b
whole <- a INTEGER DIVISION b
remainder <- a MODULUS b
power <- x EXP y
```

3.3 Comparison Operators

==
!=
<
>
<=
>=

Examples:

```
x == y
x != y
x < y
x >= y
```

3.4 Boolean Operators

NOT
AND
OR
XOR

Examples:

```
x > 0 AND x < 10
NOT finished
left OR right
flag1 XOR flag2
```

3.5 Operator Precedence

Operators are evaluated in the following order, from highest to lowest:

1. Parentheses
2. Function and collection access
3. EXP
4. Unary minus

5. *, /, INTEGER DIVISION, MODULUS
6. +, -
7. ==, !=, <, >, <=, >=
8. NOT
9. AND
10. XOR
11. OR

Parentheses should be used where ambiguity may occur.

---

4. Data Types and Literals

4.1 Supported Primitive Types

INTEGER
FLOAT
STRING
CHAR
BOOLEAN

4.2 Integer Literal

INTEGER(5)
INTEGER(-20)

Formal rule:

```
typed_integer ::= "INTEGER" , "(" , integer_literal , ")" ;
```

4.3 Float Literal

FLOAT(3.14)
FLOAT(-0.5)

Formal rule:

```
typed_float ::= "FLOAT" , "(" , float_literal , ")" ;
```

4.4 Tuple Literal

TUPLE(a, b)
TUPLE(INTEGER(1), STRING("A"))

Formal rule:

```
tuple_literal ::=
   "TUPLE" ,
   "(" ,
   [ expression , { "," , expression } ] ,
```

```
")" ;
```

4.5 Homogeneous Typed Lists

A homogeneous list contains values of one declared type.

Examples:

```
LIST OF INTEGER [1, 2, 3]
LIST OF STRING [STRING("A"), STRING("B")]
LIST OF BOOLEAN [BOOLEAN(TRUE), BOOLEAN(FALSE)]
```

Formal rule:

```
typed_list_literal ::=
   "LIST OF" ,
   type_name ,
   "[" ,
   [ expression , { "," , expression } ] ,
   "]" ;
```

All values must be compatible with the declared type.

4.6 Mixed Lists

Mixed lists use the LIST keyword.

Example:

```
LIST [INTEGER(5), STRING("hello"), CHAR("a"), BOOLEAN(TRUE)]
```

Formal rule:

```
mixed_list_literal ::=
   "LIST" ,
   "[" ,
   [ expression , { "," , expression } ] ,
   "]" ;
```

4.7 Empty Values

Supported empty values are:

```
EMPTY LIST
EMPTY DICTIONARY
EMPTY STRING
EMPTY SET
EMPTY TUPLE
```

Formal rule:

```
empty_literal ::=
     "EMPTY LIST"
   | "EMPTY DICTIONARY"
   | "EMPTY STRING"
   | "EMPTY SET"
   | "EMPTY TUPLE" ;
```

---

5. Program Structure

5.1 Program

A program contains one or more declarations or statements.

```
program ::= { declaration | statement | comment } ;
```

A program should normally contain at least one function when it is intended for automated testing.

5.2 Function Definition

Canonical syntax:

```
FUNCTION name ACCEPTS(parameter1, parameter2)
   statements
ENDFUNCTION
```

Example:

```
FUNCTION calculate_total ACCEPTS(numbers)
   total <- 0

   FOR EACH number IN numbers DO
      total <- total + number
   ENDFOR

   RETURN total
ENDFUNCTION
```

Formal rule:

```
function_definition ::=
   "FUNCTION" ,
   identifier ,
   "ACCEPTS" ,
   "(" ,
   [ parameter_list ] ,
   ")" ,
   newline ,
   { statement | comment } ,
   "ENDFUNCTION" ,
   newline ;
```

5.3 Parameter List

```
parameter_list ::=
   parameter ,
   { "," , parameter } ;
```

A parameter may optionally declare a type.

Examples:

```
FUNCTION process ACCEPTS(value)
FUNCTION process ACCEPTS(value AS INTEGER)
```

Formal rule:

```
parameter ::=
   identifier ,
   [ "AS" , type_name ] ;
```

5.4 Return Statement

```
RETURN expression
```

or:

```
RETURN
```

Formal rule:

```
return_statement ::=
   "RETURN" ,
   [ expression ] ,
   newline ;
```

A value-less return is permitted.

5.5 Procedure and Function Calls

A call may be used as an expression:

```
result <- CALL calculate_total(numbers)
```

or as a standalone statement:

```
CALL display_result(result)
```

Formal rules:

```
call_expression ::=
   "CALL" ,
   identifier ,
   "(" ,
```

```
   [ argument_list ] ,
   ")" ;

call_statement ::=
   call_expression ,
   newline ;

argument_list ::=
   expression ,
   { "," , expression } ;
```

---

6. Input and Output

6.1 Input

Input must contain a prompt expression.

```
age <- INPUT(STRING("Enter your age"))
```

Formal rule:

```
input_expression ::=
   "INPUT" ,
   "(" ,
   expression ,
   ")" ;
```

6.2 Output

```
OUTPUT expression
```

Examples:

```
OUTPUT result
OUTPUT STRING("The value is {value}")
```

Formal rule:

```
output_statement ::=
   "OUTPUT" ,
   expression ,
   newline ;
```

String placeholders use braces:

```
STRING("The result is {result}")
```

The referenced identifier must exist in the current scope.

---

7. Assignment Rules

7.1 Standard Assignment

identifier <- expression

Example:

count <- 0

7.2 Collection Element Assignment

list[index] <- value
dictionary[key] <- value

Examples:

numbers[0] <- 10
visited[TUPLE(nr, nc)] <- TUPLE(r, c)

Formal rule:

```
assignment_statement ::=
   assignable ,
   "<-" ,
   expression ,
   newline ;

assignable ::=
     identifier
   | indexed_access ;
```

---

8. Expressions

8.1 Expression Grammar

```
expression ::= or_expression ;

or_expression ::=
   xor_expression ,
   { "OR" , xor_expression } ;

xor_expression ::=
   and_expression ,
   { "XOR" , and_expression } ;

and_expression ::=
   not_expression ,
   { "AND" , not_expression } ;

not_expression ::=
     "NOT" , not_expression
   | comparison_expression ;

comparison_expression ::=
   additive_expression ,
```

```
   [ comparison_operator , additive_expression ] ;

additive_expression ::=
   multiplicative_expression ,
   { ( "+" | "-" ) , multiplicative_expression } ;

multiplicative_expression ::=
   power_expression ,
   {
      (
         "*"
       | "/"
       | "INTEGER DIVISION"
       | "MODULUS"
      ) ,
      power_expression
   } ;

power_expression ::=
   unary_expression ,
   [ "EXP" , power_expression ] ;

unary_expression ::=
     "-" , unary_expression
   | primary_expression ;

primary_expression ::=
     literal
   | identifier
   | indexed_access
   | call_expression
   | input_expression
   | length_expression
   | join_expression
   | dictionary_get_expression
   | random_expression
   | "(" , expression , ")" ;
```

8.2 Comparison Operator

```
comparison_operator ::=
     "=="
   | "!="
   | "<"
   | ">"
   | "<="
   | ">=" ;
```

---

9. Conditional Rules

9.1 IF Statement

```
IF condition THEN
   statements
ENDIF
```

Formal rule:

```
if_statement ::=
   "IF" ,
   expression ,
   "THEN" ,
   newline ,
   { statement | comment } ,
   { else_if_clause } ,
   [ else_clause ] ,
   "ENDIF" ,
   newline ;
```

9.2 ELSE IF Clause

```
ELSE IF condition THEN
   statements
```

Formal rule:

```
else_if_clause ::=
   "ELSE IF" ,
   expression ,
   "THEN" ,
   newline ,
   { statement | comment } ;
```

9.3 ELSE Clause

```
ELSE
   statements
```

Formal rule:

```
else_clause ::=
   "ELSE" ,
   newline ,
   { statement | comment } ;
```

9.4 Full Example

```
IF score >= 70 THEN
   grade <- STRING("Distinction")
ELSE IF score >= 60 THEN
   grade <- STRING("Merit")
ELSE
   grade <- STRING("Pass")
ENDIF
```

---

10. Loop Rules

10.1 WHILE Loop

```
WHILE condition DO
   statements
ENDWHILE
```

Formal rule:

```
while_statement ::=
   "WHILE" ,
   expression ,
   "DO" ,
   newline ,
   { statement | comment } ,
   "ENDWHILE" ,
   newline ;
```

10.2 Range Loop

Range loops are end-exclusive.

Therefore:

```
FOR i <- IN RANGE 0 TO n DO
```

iterates through:

```
0, 1, 2, ..., n - 1
```

Canonical syntax:

```
FOR identifier <- IN RANGE start TO end DO
   statements
ENDFOR
```

Optional step:

```
FOR i <- IN RANGE 10 TO 0 STEP -1 DO
   statements
ENDFOR
```

Formal rule:

```
range_for_statement ::=
   "FOR" ,
   identifier ,
   "<-" ,
   "IN RANGE" ,
   expression ,
   "TO" ,
   expression ,
   [ "STEP" , expression ] ,
   "DO" ,
   newline ,
   { statement | comment } ,
   "ENDFOR" ,
```

```
  newline ;
```

A step value must not equal zero.

10.3 FOR EACH Loop

```
FOR EACH item IN collection DO
  statements
ENDFOR
```

Formal rule:

```
for_each_statement ::=
  "FOR EACH" ,
  identifier ,
  "IN" ,
  expression ,
  "DO" ,
  newline ,
  { statement | comment } ,
  "ENDFOR" ,
  newline ;
```

10.4 BREAK

```
BREAK
```

Formal rule:

```
break_statement ::=
  "BREAK" ,
  newline ;
```

BREAK is valid only inside:

```
WHILE
FOR
FOR EACH
```

10.5 CONTINUE

The dialect includes an optional CONTINUE statement.

```
CONTINUE
```

Formal rule:

```
continue_statement ::=
  "CONTINUE" ,
  newline ;
```

CONTINUE is valid only inside a loop.

11. List and Collection Rules

11.1 Indexing

collection[index]

Examples:

numbers[0]
grid[row][column]

Formal rule:

```
indexed_access ::=
   primary_identifier ,
   { "[" , expression , "]" } ;

primary_identifier ::= identifier ;
```

Indexes begin at zero.

11.2 Length

LENGTH OF collection

Formal rule:

```
length_expression ::=
   "LENGTH OF" ,
   expression ;
```

11.3 Append

APPEND value TO collection

Formal rule:

```
append_statement ::=
   "APPEND" ,
   expression ,
   "TO" ,
   expression ,
   newline ;
```

11.4 Insert

INSERT value INTO collection AT index

Formal rule:

```
insert_statement ::=
   "INSERT" ,
   expression ,
   "INTO" ,
   expression ,
   "AT" ,
   expression ,
   newline ;
```

11.5 Remove First Item

REMOVE FIRST ITEM FROM collection

Formal rule:

```
remove_first_statement ::=
   "REMOVE FIRST ITEM FROM" ,
   expression ,
   newline ;
```

11.6 Remove Last Item

REMOVE LAST ITEM FROM collection

Formal rule:

```
remove_last_statement ::=
   "REMOVE LAST ITEM FROM" ,
   expression ,
   newline ;
```

11.7 Remove Specific Item

REMOVE value FROM collection

Formal rule:

```
remove_value_statement ::=
   "REMOVE" ,
   expression ,
   "FROM" ,
   expression ,
   newline ;
```

11.8 Join

JOIN(collection, separator)

Example:

```
result <- JOIN(characters, EMPTY STRING)
```

Formal rule:

```
join_expression ::=
   "JOIN" ,
   "(" ,
   expression ,
   "," ,
   expression ,
   ")" ;
```

11.9 Two-Dimensional List Creation

Canonical form:

```
grid <- EMPTY LIST
r <- 0

WHILE r < rows DO
   row <- EMPTY LIST
   c <- 0

   WHILE c < cols DO
      APPEND fill_value TO row
      c <- c + 1
   ENDWHILE

   APPEND row TO grid
   r <- r + 1
ENDWHILE
```

No special two-dimensional list syntax is required. Nested lists are constructed using ordinary list operations.

---

12. Set Rules

12.1 Set Literal

```
SET [value1, value2, value3]
```

Formal rule:

```
set_literal ::=
   "SET" ,
   "[" ,
   [ expression , { "," , expression } ] ,
   "]" ;
```

12.2 Set Operations

```
UNION(set1, set2)
INTERSECTION(set1, set2)
DIFFERENCE(set1, set2)
```

Formal rules:

```
set_operation ::=
     "UNION" , "(" , expression , "," , expression , ")"
   | "INTERSECTION" , "(" , expression , "," , expression , ")"
   | "DIFFERENCE" , "(" , expression , "," , expression , ")" ;
```

12.3 Membership

```
value IN collection
value NOT IN collection
```

These are treated as comparison expressions.

---

13. Dictionary Rules

13.1 Dictionary Creation

```
visited <- EMPTY DICTIONARY
```

13.2 Dictionary Assignment

```
dictionary[key] <- value
```

Example:

```
visited[TUPLE(nr, nc)] <- TUPLE(r, c)
```

13.3 Dictionary Retrieval

Canonical syntax:

```
value <- dictionary~GET(key)
```

Example:

```
node <- visited~GET(node)
```

Formal rule:

```
dictionary_get_expression ::=
   identifier ,
   "~GET" ,
   "(" ,
```

```
  expression ,
  ")" ;
```

13.4 Dictionary Retrieval With Default

Optional syntax:

```
value <- dictionary~GET(key, default_value)
```

Formal rule:

```
dictionary_get_expression ::=
  identifier ,
  "~GET" ,
  "(" ,
  expression ,
  [ "," , expression ] ,
  ")" ;
```

Without a default value, a missing key produces a runtime error.

13.5 Dictionary Membership

```
key IN dictionary
key NOT IN dictionary
```

---

14. Random Rules

14.1 Random Integer

```
RANDOM INT(minimum, maximum)
```

Both endpoints are inclusive.

Formal rule:

```
random_integer_expression ::=
  "RANDOM INT" ,
  "(" ,
  expression ,
  "," ,
  expression ,
  ")" ;
```

14.2 Random Float

```
RANDOM FLOAT(minimum, maximum)
```

Formal rule:

```
random_float_expression ::=
   "RANDOM FLOAT" ,
   "(" ,
   expression ,
   "," ,
   expression ,
   ")" ;
```

14.3 Random Boolean

```
RANDOM BOOLEAN()
```

Formal rule:

```
random_boolean_expression ::=
   "RANDOM BOOLEAN" ,
   "(" ,
   ")" ;
```

14.4 Combined Rule

```
random_expression ::=
     random_integer_expression
   | random_float_expression
   | random_boolean_expression ;
```

---

15. Error Handling

15.1 TRY-CATCH

```
TRY
   statements
CATCH error
   statements
ENDTRY
```

The error identifier is optional.

```
TRY
   statements
CATCH
   statements
ENDTRY
```

Formal rule:

```
try_statement ::=
   "TRY" ,
   newline ,
   { statement | comment } ,
```

```
  "CATCH" ,
  [ identifier ] ,
  newline ,
  { statement | comment } ,
  "ENDTRY" ,
  newline ;
```

15.2 RAISE

Optional explicit error generation:

```
RAISE expression
```

Example:

```
RAISE STRING("Invalid input")
```

Formal rule:

```
raise_statement ::=
  "RAISE" ,
  expression ,
  newline ;
```

---

16. Class Rules

Class support is included but may be excluded from the first experimental subset if the 50 algorithms do not require it.

16.1 Class Definition

```
CLASS ClassName
  declarations
ENDCLASS
```

Formal rule:

```
class_definition ::=
  "CLASS" ,
  identifier ,
  newline ,
  { constructor_definition | method_definition | property_declaration | comment } ,
  "ENDCLASS" ,
  newline ;
```

16.2 Constructor

The dialect uses THIS instead of Python's self.

```
CONSTRUCTOR ACCEPTS(x, y)
  PROPERTY THIS.x <- x
```

```
    PROPERTY THIS.y <- y
ENDCONSTRUCTOR
```

Formal rule:

```
constructor_definition ::=
    "CONSTRUCTOR" ,
    "ACCEPTS" ,
    "(" ,
    [ parameter_list ] ,
    ")" ,
    newline ,
    { statement | property_assignment | comment } ,
    "ENDCONSTRUCTOR" ,
    newline ;
```

16.3 Property Declaration

```
PROPERTY name
```

or:

```
PROPERTY name AS INTEGER
```

Formal rule:

```
property_declaration ::=
    "PROPERTY" ,
    identifier ,
    [ "AS" , type_name ] ,
    newline ;
```

16.4 Property Assignment

```
PROPERTY THIS.name <- value
```

Formal rule:

```
property_assignment ::=
    "PROPERTY" ,
    "THIS" ,
    "." ,
    identifier ,
    "<-" ,
    expression ,
    newline ;
```

16.5 Method Definition

```
METHOD name ACCEPTS(parameters)
    statements
ENDMETHOD
```

Formal rule:

```
method_definition ::=
   "METHOD" ,
   identifier ,
   "ACCEPTS" ,
   "(" ,
   [ parameter_list ] ,
   ")" ,
   newline ,
   { statement | comment } ,
   "ENDMETHOD" ,
   newline ;
```

16.6 Property Access

```
THIS.property_name
object.property_name
```

Formal rule:

```
property_access ::=
   identifier ,
   "." ,
   identifier ;
```

THIS is treated as a reserved contextual identifier inside classes.

---

17. Type Names

```
type_name ::=
     "INTEGER"
   | "FLOAT"
   | "STRING"
   | "CHAR"
   | "BOOLEAN"
   | "LIST"
   | "DICTIONARY"
   | "SET"
   | "TUPLE"
   | identifier ;
```

A user-defined class name may be used as a type.

---

18. Statement Grammar

```
statement ::=
     assignment_statement
   | output_statement
   | call_statement
   | append_statement
```

```
   | insert_statement
   | remove_first_statement
   | remove_last_statement
   | remove_value_statement
   | if_statement
   | while_statement
   | range_for_statement
   | for_each_statement
   | return_statement
   | break_statement
   | continue_statement
   | try_statement
   | raise_statement ;
```

---

19. Declaration Grammar

```
declaration ::=
     function_definition
   | class_definition ;
```

---

20. Literal Grammar

```
literal ::=
     integer_literal
   | float_literal
   | typed_integer
   | typed_float
   | string_literal
   | char_literal
   | boolean_literal
   | tuple_literal
   | typed_list_literal
   | mixed_list_literal
   | set_literal
   | empty_literal ;
```

---

21. Complete High-Level Grammar

```
program ::=
   { declaration | statement | comment } ;

declaration ::=
     function_definition
   | class_definition ;

statement ::=
     assignment_statement
   | output_statement
   | call_statement
   | append_statement
   | insert_statement
   | remove_first_statement
   | remove_last_statement
   | remove_value_statement
```

```
    | if_statement
    | while_statement
    | range_for_statement
    | for_each_statement
    | return_statement
    | break_statement
    | continue_statement
    | try_statement
    | raise_statement ;

expression ::=
    or_expression ;

or_expression ::=
    xor_expression ,
    { "OR" , xor_expression } ;

xor_expression ::=
    and_expression ,
    { "XOR" , and_expression } ;

and_expression ::=
    not_expression ,
    { "AND" , not_expression } ;

not_expression ::=
      "NOT" , not_expression
    | comparison_expression ;

comparison_expression ::=
    additive_expression ,
    [
      (
        "=="
      | "!="
      | "<"
      | ">"
      | "<="
      | ">="
      | "IN"
      | "NOT IN"
      ) ,
      additive_expression
    ] ;

additive_expression ::=
    multiplicative_expression ,
    { ( "+" | "-" ) , multiplicative_expression } ;

multiplicative_expression ::=
    power_expression ,
    {
      (
        "*"
      | "/"
      | "INTEGER DIVISION"
      | "MODULUS"
      ) ,
      power_expression
    } ;

power_expression ::=
    unary_expression ,
```

```
   [ "EXP" , power_expression ] ;

unary_expression ::=
     "-" , unary_expression
   | primary_expression ;

primary_expression ::=
     literal
   | identifier
   | indexed_access
   | property_access
   | call_expression
   | input_expression
   | length_expression
   | join_expression
   | dictionary_get_expression
   | random_expression
   | set_operation
   | "(" , expression , ")" ;
```

---

22. Static Semantic Rules

The parser may produce a syntactically valid AST that still violates semantic rules. The following checks must therefore be performed after parsing.

22.1 Identifier Rules

An identifier must be declared or assigned before it is read.

Invalid:

```
OUTPUT total
total <- 0
```

22.2 Scope Rules

Variables declared within a function belong to that function.

Loop variables belong to the containing function or block, depending on the implementation policy.

Function parameters are available throughout the function body.

22.3 Type Compatibility

Operations must use compatible types.

Valid:

```
total <- INTEGER(5) + INTEGER(2)
```

Invalid:

```
total <- INTEGER(5) + STRING("two")
```

22.4 Boolean Conditions

Conditions used by IF and WHILE must evaluate to Boolean values.

22.5 Return Rules

A RETURN statement may only occur inside a function, method, or constructor where explicitly permitted.

Constructors should not return a value.

22.6 Break and Continue Rules

BREAK and CONTINUE may only occur inside loops.

22.7 Collection Rules

List indexes must be integers.

Dictionary keys must be hashable in the generated Python representation.

Collection operations must target compatible collection types.

22.8 Function Call Rules

The number of supplied arguments must match the number of required parameters unless optional parameters are later added.

22.9 Range Rules

The STEP value must not be zero.

Range loops are end-exclusive.

22.10 Character Rules

A CHAR value must contain exactly one character or one valid escape sequence.

---

23. Normalisation Rules

The evaluation framework may perform conservative normalisation before strict parsing.

Permitted transformations include:

Keyword case normalisation
Unicode arrow → <-
Removal of Markdown code fences
Line-ending normalisation
Trailing whitespace removal
Approved spelling correction
Approved synonym mapping

Examples:

```
while x < 5 do
```

may become:

```
WHILE x < 5 DO
x ← 5
```

may become:

```
x <- 5
```

The framework must log every transformation.

It must not:

invent missing variables;
insert missing algorithmic operations;
change conditions;
add missing branches;
alter arithmetic operators;
repair semantic logic silently.

---

24. Guided Link Rules

Guided Links are documentation annotations and do not alter program execution.

Recommended syntax:

```
GUIDED LINK("typed lists")
```

Formal rule:

```
guided_link ::=
   "GUIDED LINK" ,
   "(" ,
   quoted_string ,
   ")" ,
   newline ;
```

Guided Links may appear between statements and should be:

- stored as metadata;
- excluded from AST execution nodes;
- excluded from generated Python;
- optionally shown in teaching-oriented visualisations.

---

25. Complete Example

```
FUNCTION find_largest ACCEPTS(numbers AS LIST)
   IF LENGTH OF numbers == 0 THEN
      RAISE STRING("The list cannot be empty")
   ENDIF

   largest <- numbers[0]

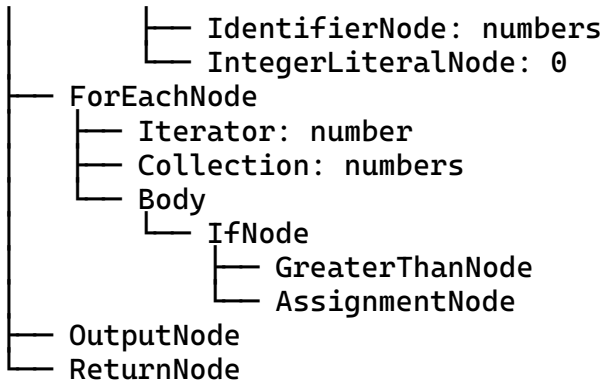
   FOR EACH number IN numbers DO
      IF number > largest THEN
         largest <- number
      ENDIF
   ENDFOR

   OUTPUT STRING("The largest value is {largest}")
   RETURN largest
ENDFUNCTION
```

---

26. Expected AST Outline

```
ProgramNode
└── FunctionNode: find_largest
    ├── ParameterNode: numbers AS LIST
    └── Body
        ├── IfNode
        │   ├── Condition
        │   │   └── EqualNode
        │   │       ├── LengthNode
        │   │       │   └── IdentifierNode: numbers
        │   │       └── IntegerLiteralNode: 0
        │   └── ThenBody
        │       └── RaiseNode
        │           └── StringLiteralNode
        ├── AssignmentNode
        │   ├── IdentifierNode: largest
        │   └── IndexAccessNode
```



---

27. Versioning Rule

This specification is frozen as:

Cambridge-Style Pseudocode — Alan_24038155 Dialect v1.0

Any later syntax addition, removal, or semantic alteration must create a new version, for example:

AJ_24038155 Dialect v1.1
AJ_24038155 Dialect v2.0

AI outputs should always be evaluated against the exact grammar version recorded at generation and evaluation time.